スポットインスタンスにおける
期限条件付き完了可能性に基づくスケジューリング戦略

—- 数式モデルと実験設計 —-

那須 渚

2026 年 10 月 12 日

概要

本メモは，既発表の測定フレームワークで定義した期限条件付き完了可能性 $F_{\mathrm{comp}}(d)$ を**入力**として用い，短時間ワークロードの実行先（リージョン・インスタンスタイプ・起動時刻）を決定するスケジューリング戦略を数式モデルとして定式化する．あわせて，GitHub Actions 上で戦略あり・なしを比較する実験設計を示す．本メモで必要となる量は p（$= F_{\mathrm{comp}}$），$\bar{u}$（中断時の平均稼働時間），P_{spot}（スポット価格）の 3 つのみであり，いずれも既存の観測データから直接算出できる．

目次

1 **背景と本メモの位置づけ** 1

2 **戦略の数式モデル** 1

2.1 計算したいもの . . . 1
2.2 記号の定義 . . . 2
2.3 ステップ 1：何回実行すれば完了するか . . . 2
2.4 ステップ 2：合計で何分課金されるか . . . 3
2.5 ステップ 3：コストへの換算 . . . 3
2.6 ステップ 4：最適な実行条件の選択 . . . 4
2.7 ステップ 5：制約条件 . . . 4
2.8 最適化問題としての定式化 . . . 4
2.9 実装 . . . 5

3 **比較対象とする戦略** 6

4 **推定値の不確実性の扱い** 7

5 **実験設計：GitHub Actions** 7

5.1 構成 . . . 7
5.2 実験パラメータ . . . 8
5.3 評価指標 . . . 8

6 まとめ 8

1 背景と本メモの位置づけ

既発表の内容は，測定フレームワークの提案と観測結果の報告にとどまっている．すなわち $F_{\mathrm{comp}}(d)$ を定義し，時間帯による変動（Night 期間が最も高い）を示したが，**それを用いて実際に何かを決定する仕組み**を与えていない．

次の一歩は明確である．

> $F_{\mathrm{comp}}(d)$ を**入力**とするスケジューリング戦略を数式で定義し，GitHub Actions の実ジョブ上で**戦略あり**と**戦略なし**を比較する．

これにより，研究の位置づけが「観測研究」から「提案手法 + 評価」へ移行する．

2 戦略の数式モデル

2.1 計算したいもの

本モデルが計算したいのは次の一点に尽きる．

> **中断されるとやり直しになるとき，ジョブ 1 本を完了させるのに結局いくらかかるか．**

1 回で成功すればその分だけ支払う．失敗すれば，失敗した試行の料金も支払った上で再度実行する．この「結局いくら」を定式化する．

2.2 記号の定義

ジョブ j は時刻 t_0 に到着し，次の 4 つで特徴づけられる．

表 1 ジョブの特徴量

記号	名称	意味
d_j	実行所要時間	ジョブが必要とする実行時間（分）．$5, 10, 30, 60$
D_j	許容遅延	$t_0 + D_j$ までに完了すればよい，という締切
$\mathcal{T}_j$	利用可能タイプ	使用してよいインスタンスタイプの集合
$\mathcal{R}_j$	利用可能リージョン	使用してよいリージョンの集合（データ所在地制約など）

決定変数は次の 3 つ組である．

$$x = (r,\ \tau,\ t) \qquad \begin{array}{l} r : \text{リージョン} \\ \tau : \text{インスタンスタイプ} \\ t : \text{起動時刻} \end{array} \tag{1}$$

2.3 ステップ 1：何回実行すれば完了するか

p を 1 回の試行が成功する確率とする．これは観測済みの $F_{\mathrm{comp}}(d)$ そのものである．

$$p(r,\tau,t,d) \;=\; \hat{F}_{\mathrm{comp}}^{(r,\tau,\pi(r,t))}(d) \tag{2}$$

ここで $\pi(r,t)$ は，リージョン r の現地時刻における時間帯区分

$$\pi(r,t) \in \{\mathrm{Night},\ \mathrm{Morning},\ \mathrm{Afternoon},\ \mathrm{Evening}\}$$

を表す．

成功するまで繰り返すとき，試行回数は幾何分布に従い，その期待値は

$$\boxed{\mathbb{E}[\text{試行回数}] \;=\; \frac{1}{p}} \tag{3}$$

となる．直感的には「10 回中 7 回成功するなら，1 回成功させるのに 10/7 回必要」ということである．

表 2　成功確率と期待試行回数（観測値の例）

条件	時間帯	p	$1/p$
N. Virginia / c6a.large / $d = 60$	Night	0.98	1.02 回
N. Virginia / c6a.large / $d = 60$	Morning	0.69	1.45 回

2.4 ステップ 2：合計で何分課金されるか

試行は 2 種類に分かれる．

成功した試行（必ず最後の 1 回）　d 分すべて実行して完了するため，d **分**課金される．
失敗した試行（平均 $\frac{1}{p} - 1$ 回）　途中で中断されるため，中断までの時間だけ課金される．

ここで $\bar{u}$ を，**中断された試行が平均して何分稼働してから停止したか**とする．これは中断された事象に条件付けた打ち切り期待値である．

$$\bar{u}(r,\tau,\pi,d) \;=\; \mathbb{E}[\,U \mid U < d\,] \tag{4}$$

U は観測された稼働時間 `uptime_s` であり，既存データから直接算出できる．

以上より，期待課金時間は次式で与えられる．

$$\boxed{\mathbb{E}[\text{課金時間}] \;=\; \underbrace{d}_{\substack{\text{成功した}\\ 1\text{ 回}}} \;+\; \underbrace{\left(\frac{1}{p} - 1\right)}_{\substack{\text{失敗した}\\ \text{回数}}} \times \underbrace{\bar{u}}_{\substack{1\text{ 回あたりの}\\ \text{無駄時間}}}} \tag{5}$$

■数値例. $d = 60$ 分，$\bar{u} \approx 12$ 分として，

$$\text{Night } (p = 0.98): \quad 60 + \left(\tfrac{1}{0.98} - 1\right) \times 12 = 60 + 0.020 \times 12 = 60.2 \text{ 分} \tag{6}$$

$$\text{Morning } (p = 0.69): \quad 60 + \left(\tfrac{1}{0.69} - 1\right) \times 12 = 60 + 0.449 \times 12 = 65.4 \text{ 分} \tag{7}$$

すなわち Morning は Night に比べて約 5.2 分ぶん余計に課金される．これが時間帯を選択することの金銭的な意味である．

2.5 ステップ 3：コストへの換算

$P_{\text{spot}}(r, \tau)$ を 1 時間あたりのスポット価格（実測値）とすると，

$$\boxed{C(r, \tau, t \mid d) \;=\; \frac{P_{\text{spot}}(r, \tau)}{60} \;\times\; \mathbb{E}[\text{課金時間}]} \tag{8}$$

60 で除しているのは，価格が**時間単位**，課金時間が**分単位**であり，単位を揃えるためである．

■数値例. $P_{\text{spot}} = 0.0295$ USD/h（us-east-1 の c6a.large）のとき，

$$\text{Night}: \quad \frac{0.0295}{60} \times 60.2 = 0.0296 \text{ USD} \tag{9}$$

$$\text{Morning}: \quad \frac{0.0295}{60} \times 65.4 = 0.0322 \text{ USD} \tag{10}$$

2.6 ステップ 4：最適な実行条件の選択

ここまでで，(r, τ, t) を決めればコストが計算できる状態になった．あとは最も安い組み合わせを選べばよい．

$$(r^*,\ \tau^*,\ t^*) \;=\; \underset{r,\ \tau,\ t}{\arg\min}\ C(r, \tau, t \mid d_j) \tag{11}$$

arg min は「最小値そのもの」ではなく，**最小値を与える** (r, τ, t) を意味する．「すべての組み合わせを調べ，最も安かったものを選ぶ」と読み替えてよい．

候補数は

$$4 \text{ リージョン} \times 3 \text{ タイプ} \times 24 \text{ 時刻} = 288 \text{ 通り}$$

であり，全列挙で即座に解ける．ヒューリスティクスは不要である．

2.7 ステップ 5：制約条件

単に安いだけを求めると「来週の深夜に実行する」といった解が得られてしまう．そこで制約を課す．

■締切制約．t は次の範囲からのみ選べる．

$$t \in \left[\, t_0,\ \ t_0 + D_j - d_j \,\right] \tag{12}$$

D_j は「いつまでに終わればよいか」，d_j は実行所要時間であるから，**遅延させてよい上限**は $D_j - d_j$ となる．この「待てる幅」を **slack** と呼ぶ．

> 例：10 分のジョブで「6 時間以内に完了すればよい」場合，$D_j - d_j = 6\,\mathrm{h} - 10\,\mathrm{min} \approx 5$ 時間 50 分まで待てる．

slack $= 0$ のときは $t = t_0$ の一点に退化し，選択できるのはリージョンとタイプのみとなる．**これは既発表の “region selection can substitute for time-shifting” を数式で表現したものである．**

2.8 最適化問題としての定式化

以上をまとめると，戦略は次の制約付き最適化問題として記述できる．

$$\begin{aligned}
\min_{r,\ \tau,\ t} \quad & C(r, \tau, t \mid d_j) && \text{(期待コスト)} \\
\text{s.t.} \quad & \mathbb{E}\big[T_{\text{finish}}(r, \tau, t)\big] \ \le\ D_j && \text{(締切制約)} \\
& p(r, \tau, t, d_j) \ \ge\ p_{\min} && \text{(信頼性下限)} \\
& r \in \mathcal{R}_j, \quad \tau \in \mathcal{T}_j && \text{(選択肢の範囲)} \\
& t \in \left[\, t_0,\ t_0 + D_j - d_j \,\right] && \text{(時間シフトの範囲)}
\end{aligned} \tag{13}$$

`s.t.` は *subject to*（〜 という条件のもとで）の略である．式 (13) は日本語に直せば次の一文にすぎない．

> **締切・信頼性・選択肢の条件をすべて満たしつつ，期待コストが最小となる (r, τ, t) を求めよ．**

2.8.1 各行の意味

■1 行目：目的関数． min の下に添えられた r, τ, t は，**動かしてよいのはこの 3 つだけ**であることを示す（決定変数）．$C(\cdot \mid d_j)$ の縦棒は条件付きを表し，d_j は選択対象ではなくジョブの性質として所与であることを意味する．10 分のジョブは 10 分かかり，これは変更できない．変更できるのは**どこで・どのタイプで・いつ**実行するかの 3 つのみである．

■2 行目：締切制約． T_{finish} はジョブの終了時刻，$\mathbb{E}[\cdot]$ はその期待値である．中断されると再試行により終了時刻が延びるため，終了時刻は確率的に変動する．よって期待値で評価する．近似的には

$$\mathbb{E}\big[T_{\text{finish}}\big] \ \approx\ \underbrace{(t - t_0)}_{\substack{\text{起動を} \\ \text{待った時間}}} \ +\ \underbrace{\frac{1}{p}\,(s + d)}_{\substack{\text{再試行を含む} \\ \text{実行時間}}} \tag{14}$$

ここで s は起動オーバーヘッドであり，本データでは実測で約 4.5 秒（`startup_s` の平均）である．この制約により「安価だが中断が頻発し，結果として締切に間に合わない」選択肢が排除される．

■3 行目：信頼性下限．成功確率が $p_{\min}$ 未満のセルは候補から除外する．

締切制約があるにもかかわらずこの制約が必要な理由は，**期待値が平均値にすぎない**ためである．$p = 0.3$ のセルであっても，d が短ければ再試行の繰り返しにより「平均すれば間に合う」という計算結果になり得る．しかし実務上，3 回に 2 回失敗する CI は利用に耐えない．分散が大きすぎる．$p_{\min}$ は，期待値では表現できない運用上の安心感を直接制約として与えるものである．

実験では $p_{\min} \in \{0,\ 0.8,\ 0.9\}$ と変化させ，**どの程度保守的であるべきか**自体を結果として示す（$p_{\min} = 0$ は制約なしに相当する）．

■4 行目：選択肢の範囲．$\in$ は「$\sim$ の中から選ぶ」ことを表す．

- $r \in \mathcal{R}_j$：すべて利用可能なら 4 リージョン．「データを EU 外に出せない」なら $\mathcal{R}_j = \{\text{eu-west-1}\}$ の 1 点．
- $\tau \in \mathcal{T}_j$：GPU が必須なら $\mathcal{T}_j = \{\text{g4dn.xlarge}\}$．任意なら 3 タイプすべて．
- $t \in [\,t_0,\ t_0 + D_j - d_j\,]$：左端は「直ちに実行」，右端は「これ以上遅らせると締切に間に合わない限界」．[ ] は両端を含む閉区間を表す．

2.9 実装

式 (13) の各行は，そのままコードの各行に対応する．

```
def solve(d, D, t0, p_min, R_allowed, T_allowed):
    best, best_cost = None, float("inf")

    for r in R_allowed:                              # <- 4 行目 r in R_j
        for tau in T_allowed:                        # <- 4 行目 tau in T_j
            for t in range(t0, t0 + D - d + 1):      # <- 4 行目 t の範囲
                p = F_comp[r, tau, period(r, t), d]

                if p < p_min:                        # <- 3 行目 信頼性下限
                    continue
                if expected_finish(r, tau, t, d) > D:    # <- 2 行目 締切
                    continue

                c = cost(r, tau, t, d)               # <- 1 行目 目的関数
                if c < best_cost:
                    best, best_cost = (r, tau, t), c

    return best                                      # <- argmin
```

- 3 重 `for` = 4 行目（探索範囲）
- 2 つの `continue` = 2 行目・3 行目（制約違反の除外）
- `if c < best_cost` = 1 行目（min）

すなわち式 (13) は，**288 通りを走査して最も安いものを選ぶ**という処理を数式として記述したものにほかならない．

■数式で記述する理由.

1. 何が変数で何が所与かが一目で判別できる．(r, τ, t) が変数であり，$d_j, D_j, p_{\min}$ は入力である．
2. 制約を加減することで戦略を体系的に定義できる（第 3 節）．
3. 査読者が短時間で理解できる．

3　比較対象とする戦略

評価設計の要点は，**式 (13) の制約を変化させるだけで各戦略を定義できる**ことにある．

表 3　比較する 5 つの戦略

戦略	定義	式 (13) における変更点
S0: Naive	$t = t_0$，r, τ 固定	t を t_0 に固定し，$\mathcal{R}_j, \mathcal{T}_j$ も 1 点に固定．GitHub Actions の素の挙動に相当
S1: Cheapest	$\arg\min P_{\text{spot}}$，即時実行	目的関数を C ではなく P_{spot} のみとする（p を無視）．価格のみを見る実務的手法
S2: Region-only	$t = t_0$ 固定，r のみ自由	slack $= 0$ の場合．時間シフトできないジョブ向け
S3: Proposed	式 (13) そのまま	(r, τ, t) すべてを最適化
S4: Oracle	実測に基づく事後最良	達成可能な上界

S2 を独立に設けることで，**時間シフトの寄与**と**リージョン選択の寄与**を分離して評価できる．これは査読において高い確度で問われる点である．

4　推定値の不確実性の扱い

$\hat{F}_{\text{comp}}$ はサンプル数 30–42 件から得た推定値であり，既発表の Limitations にも記したとおり不確かである．点推定をそのまま用いると，**偶然高い値を示したセル**を選択してしまう（winner's curse）．

対策として，Wilson の下側信頼限界を用いた保守的戦略を併せて評価する．

$$\tilde{p} \;=\; \hat{p}_{\text{LCB}} \;=\; \frac{\hat{p} + \dfrac{z^2}{2n} \;-\; z\sqrt{\dfrac{\hat{p}\,(1-\hat{p})}{n} + \dfrac{z^2}{4n^2}}}{1 + \dfrac{z^2}{n}} \tag{15}$$

式 (15) を記憶する必要はない．`statsmodels.stats.proportion.proportion_confint(k, n, method='wils` の戻り値の下側がこれにあたる．

理解しておくべき性質は次の 2 点のみである．

- n が小さいほど $\tilde{p}$ は $\hat{p}$ より大きく低下する．
- したがって，**サンプル数の少ないセルを自動的に回避する．**

既発表で 95% 信頼区間を提示しているため, そのまま接続して利用できる. $\hat{p}$ 版と $\tilde{p}$ 版（**S3-LCB**）の双方で戦略を実行して比較すれば，それ自体が一つの知見となる.

5 実験設計：GitHub Actions

5.1 構成

```
GitHub Actions workflow (matrix: strategy x deadline)
   |
   +- job: decide        <- ubuntu-latest (GitHub hosted)
   |    scheduler.py: 戦略 S に従って (r, tau, t) を決定
   |      入力: data/*.csv から構成した F_comp テーブル + 価格表
   |      出力: decision.json
   |
   +- job: provision     <- 決定した t まで待機 -> r でスポット起動
   |    self-hosted runner を登録 (ephemeral, --once)
   |
   +- job: workload      <- runs-on: [self-hosted, <run_id>]
   |    実際の CI ジョブ
   |
   +- job: collect       <- 結果を DynamoDB に記録
        成功/中断, 試行回数, 課金時間, 実コスト, wall-clock
```

■中断の検知. runner 上で IMDS の `spot/instance-action` を監視する（既存 `start_instances.py` のロジックを流用）. 検知したら直ちにジョブを失敗として記録し，`provision` から再試行する. これにより「全やり直し」の挙動を実測できる.

■実装上の注意. `jobs.<id>.timeout-minutes` と，ephemeral runner の`--ephemeral` フラグを必ず使用すること. 非 ephemeral の場合, 中断後にゾンビ runner が残存し, 次の試行が汚染される.

5.2 実験パラメータ

表 4 実験パラメータ

パラメータ	水準	備考
d_j（所要時間）	5, 10, 30, 60 分	既発表と同一
$D_j - d_j$（slack）	0, 1, 3, 6, 12 時間	0 は時間シフト不可（S2 に対応）
$p_{\min}$	0, 0.8, 0.9	0 は制約なし
戦略	S0–S4, S3-LCB	表 3

5.3 評価指標

- 実コスト（USD）：戦略ごとの実測値
- 完了までの wall-clock 時間
- 試行回数（= 中断による再実行回数）

- 締切遵守率：$T_{\text{finish}} \le D_j$ となった割合
- 予測精度：$\mathbb{E}$[課金時間] と実測課金時間の乖離

最後の項目は重要である．これはモデル (5) の妥当性検証に相当し，「式が現実を説明できているか」を直接示す．

6 まとめ

表 5　各式の要約

式	意味
$\dfrac{1}{p}$	成功するまでの平均試行回数
$d + \left(\dfrac{1}{p} - 1\right)\bar{u}$	合計の期待課金時間（分）
$\dfrac{P_{\text{spot}}}{60} \times$ (上式)	期待コスト（USD）への換算
$\arg\min_{r,\tau,t}$	288 通りを走査し最安の条件を選択
$t \le t_0 + D_j - d_j$	締切までに待てる幅（slack）
$\tilde{p}$（Wilson LCB）	サンプル数の少ないセルを信用しない

必要な量は p（$= F_{\text{comp}}$，算出済み），$\bar{u}$（中断時の `uptime_s`，データに存在），P_{spot}（実測済み）の 3 つのみであり，**すべて既存データから算出できる．**

■次の作業．既存 CSV から p，$\bar{u}$，P_{spot} の 3 テーブルを構成し，S0–S4 の戦略をシミュレーションで比較する `strategy.py` を実装するのが最短である．GitHub Actions による実機実験はその後でよい．シミュレーションで差が生じる条件を特定してから実機に移るほうが，費用と時間の双方を節約できる．